

Introduzione a Python

Python

Python è un linguaggio di programmazione di alto livello, interpretato e orientato agli oggetti, noto per la sua semplicità e leggibilità. [^]object-oriented
Per queste ragioni, Python è un linguaggio di programmazione ideale sia per principianti che per sviluppatori esperti.

Linguaggio di alto livello: I linguaggi di alto livello sono simili all'inglese e facili da apprendere e utilizzare.

Le istruzioni in un linguaggio di programmazione di alto livello sono chiamate *istruzioni* (statements).

Ecco, ad esempio, un'istruzione in un linguaggio di alto livello che calcola l'area di un cerchio:

```
area = 5 * 5 * 3.1415 .
```

Linguaggio interpretato:

Un linguaggio interpretato utilizza un programma chiamato interprete, che traduce un linguaggio di alto livello una istruzione alla volta in linguaggio macchina e poi esegue il programma.

L'interprete individuerà diversi tipi di errori e interromperà l'esecuzione del programma quando ne viene rilevato uno.

Linguaggio macchina:

Le istruzioni sono sotto forma di codice binario. Ad esempio, per sommare due numeri, potresti dover scrivere un'istruzione in codice binario, come questa:

```
1101101010011010 .
```

La Programmazione Orientata agli Oggetti (OOP):

è un paradigma di programmazione che organizza il codice in oggetti, che sono entità che combinano dati (chiamati attributi) e comportamenti (chiamati metodi), semplificando l'organizzazione del codice e rendendolo più modulare e riutilizzabile.

Le principali caratteristiche di Python includono:

- **Sintassi semplice e intuitiva:**
- facilita lo sviluppo rapido del codice.

- Supporto per i paradigmi di programmazione imperativa, funzionale e orientata agli oggetti.
 - **Ampie librerie standard:**
che offrono moduli e funzioni per operazioni comuni, come la gestione dei file, la rete, l'elaborazione dei dati e molto altro.
Grazie alla sua versatilità, Python è utilizzato in vari settori, tra cui lo sviluppo web, l'analisi dei dati, l'intelligenza artificiale, la cybersecurity, la scienza computazionale e l'automazione.
-

Programmare in Python

Un codice è una serie di istruzioni che i programmatori scrivono. Il processo di scrivere un programma è chiamato anche codifica.

Ora creeremo il nostro primo codice in Python: Crea un file chiamato helloWorld.py. Usa un editor per scrivere le istruzioni e salvale nel file helloWorld.py.

```
My first Python code
print ("Hello World")
"This program displays the message: Hello World!"
```

I commenti

I commenti aiutano i programmatori a comunicare e comprendere un programma. Non sono istruzioni di programmazione e quindi vengono ignorati dall'interprete.

In Python, i commenti sono preceduti da:

- **un simbolo cancelletto (#) su una riga (chiamato line comment):**

```
#My first Python code
```
 - Oppure racchiusi tra tre apici singoli consecutivi (""") su una o più righe (chiamato paragraph comment):

```
'''this program displays the message: Hello World!'''
```
-

Data types

Cosa sono i data types

Quando ci avviciniamo a un qualsiasi linguaggio di programmazione ci avviciniamo a dei dati primitivi

Un programma deve rappresentare diversi tipi di valori con cui lavorare. Iniziamo con i più semplici: i tipi primitivi.

Ce ne sono di diversi tipi:

- **Intero (int):**

Numeri senza decimali. Possono essere positivi o negativi.

- **Flottante (float):**

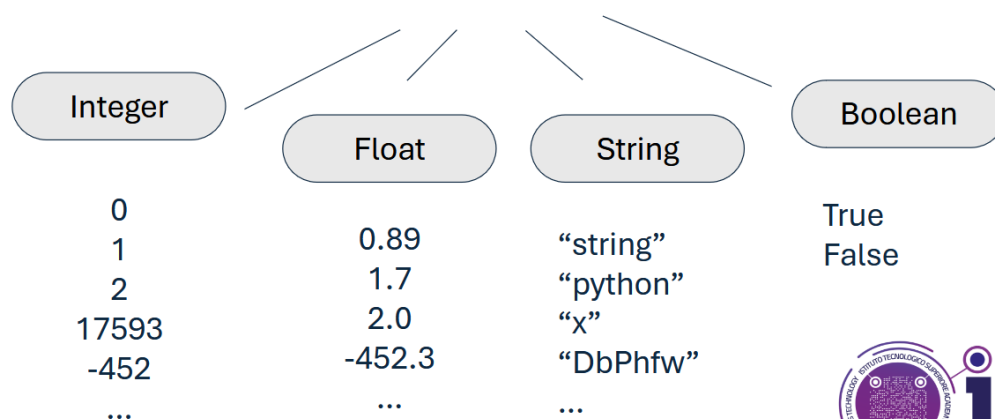
Numeri con decimali. Possono essere positivi o negativi.

- **Stringa (str):**

Dati testuali, ma in realtà possono essere qualsiasi simbolo purché siano racchiusi tra virgolette (" " o ' ').

- **Booleano (bool):**

Valore restituito dalle operazioni logiche. Può essere solo True o False (attenzione alla sensibilità delle maiuscole!).



Data types.png

I numeri possono avere qualsiasi valore come le stringhe, i valori booleani possono avere 2 valori `True` o `false`

Variabili

Una volta compreso i dati devi saperli usare, le variabili sono dei modi per salvare questi valori di dati e li puoi richiamare richiamando la variabile a cui hai assegnato quel valore (dato).

A una variabile posso dargli qualsiasi nome e valore, i nomi devono descrivere al meglio il tipo di dato che state usando (es: name, age).

Consideriamo prima il semplice problema di calcolare l'area di un cerchio.

Come scriviamo un programma per risolvere questo problema?

Scrivere un programma implica progettare algoritmi e poi tradurli in istruzioni di programmazione, ovvero codice.

Un algoritmo descrive come un problema viene risolto elencando le azioni che devono essere intraprese e l'ordine in cui devono essere eseguite.

L'algoritmo per calcolare l'area di un cerchio può essere descritto come segue:

1. Ottieni il raggio del cerchio dall'utente.
2. Calcola l'area applicando la seguente formula:
$$\text{area} = \text{raggio} \times \text{raggio} \times \pi$$
3. Visualizza il risultato.

In questo problema, il programma deve ottenere il valore del raggio.

Il valore del raggio può essere memorizzato nella memoria del computer. Per accedervi, il programma deve usare una variabile.

Una variabile è un nome che fa riferimento a un valore memorizzato nella memoria del computer.

Invece di usare x e y come nomi di variabili, è meglio scegliere nomi descrittivi :

In questo caso, ad esempio, puoi usare il nome "raggio" per la variabile che fa riferimento al valore del raggio del cerchio e "area" per la variabile che fa riferimento al valore dell'area del cerchio.

Quindi possiamo scrivere nel nostro programma computeArea.py



Python

```
1  #Assign a value to radius
2  radius=20
3  #compute area
4  area= radius * radius * 3.14
5  #display result
6  print(area)
```

Vogliamo rappresentare nel nostro programma quanto siamo vecchi:

Es:



Python

```
1  gianpino = 27
2  print (gianpino)
```

Supponiamo che ci sia un'altra persona con un'età differente con lo stesso nome, basterà solo assegnare un nuovo valore alla variabile `gianpino`.



Python

```
1 gianpino = 30
2 print(gianpino)
```

Possiamo anche modificare il data type alla nostra variabile:



Python

```
1 type(gianpino)
2 >>> <class 'int'>
```

Mentre se vogliamo assegnargli un valore con la virgola:



Python

```
1 gianpino = 27.5
2 type(gianpino)
3 >>> <class 'float'>
```

Le variabili di python sono dei puntatori non dei recipienti!

🔗 Le variabili di python sono dei pointers

In molti linguaggi di programmazione, le variabili sono come contenitori o secchi (buckets) nei quali inserisci dati.

Ad esempio in **C**:



C

```
1 int x = 4;
```

In questo caso vado a definire un "memory bucket" chiamato `x`: a cui sto inserendo dentro il valore `4`.

In python, invece, le variabili bisogna pensarle non come dei contenitori ma come dei pointers.

Es:



Python

```
1 x = 4
```

In questo caso ho definito un pointer chiamato `x` che punta ad altri bucket contenenti il valore `4`.

Una conseguenza di ciò è che, poiché le variabili in Python puntano semplicemente a vari oggetti, non è necessario "dichiarare" la variabile, né richiedere che la variabile punti sempre a informazioni dello stesso tipo.

Per questo che python viene classificato come linguaggio dinamicamente-typed(tipizzato dinamicamente):

i nomi delle variabili possono puntare a ogni tipo di oggetti.

Per fare un esempio pratico:



Python

```
1  x = 1          # x è un integer
2  x = "hello"    # x è una stringa
3  x = [1, 2, 3]  # x è una lista
```

Mentre se si utilizzassero dei linguaggi statically-typed(tipizzati staticamente) potrebbe mancare la *type-safety* (sicurezza sui tipi) offerta da dichiarazioni come quelle presenti in **C**.

Questo typing dinamico è una delle carte vincenti di Python, che lo hanno reso fin da subito un linguaggio veloce sia da scrivere che da leggere

⚠ Overwriting: Fate attenzione!

Proprio per questo motivo è semplice cadere nell'overwriting e commettere degli errori.



Python

```
1  age = 30
2  name = "Bob"
3  ...
4  name = 31 ← Qui ad esempio volevamo aggiornare l'età ma
               abbiamo accidentalmente codificato il nome. In questo caso
               Python non genera nessun errore!
5  Altri linguaggi, come Java o C++ sono tipizzati: per ogni
   variabile, devi predefinire il suo tipo di dato.
```

🔗 Type or not Type ▾

Come è stato accennato nel file "Spiegazione esercizi Lezione 1 Python" (vedi la sezione [Le variabili](#)), quando si dichiara la variabile bisogna mettergli accanto la seguente annotazione di tipo (es: `:int`, `:float`, `list`, etc). Tuttavia essendo python un linguaggio tipizzato dinamicamente, in realtà questo non è una regola che va ad influire sull'esecuzione del codice, ma più per rendere il codice chiaro agli strumenti di analisi ed a eventuali sviluppatori che stanno collaborando al progetto.

Nello specifico questo metodo presenta alcuni vantaggi e differenze con il metodo classico di assegnazione dei valori alla variabili:

Prendiamo ad esempio due variabili con due valori integer

 Python

```
1 x = 4
2 y:int = 5
3
```

Alla prima variabile `x` sto assegnando il valore `4` e in questo caso il tipo di `x` verrà automaticamente dedotto come `int`.

Mentre alla seconda variabile `y`, sto assegnando il valore `5` e sto dicendo a python, tramite l'annotazione di tipo, che `y` dovrebbe essere un intero (`:int`).

Questa annotazione in realtà non cambia il comportamento del codice runtime, ma migliora la leggibilità e la documentazione.

Quindi in pratica entrambe le versioni funzionano allo stesso modo a runtime.

La differenza tra i due è che la seconda (`y:int=5`) è utile per documentare il tipo previsto della variabile, migliorando la comprensione del codice e aiutando gli strumenti di analisi.

Es:

 Python

```
1 x = 4           # Assegnazione semplice
2 y: int = 4      # Assegnazione con annotazione di tipo
3
4 print(type(x))  # Output: <class 'int'>
5 print(type(y))  # Output: <class 'int'>>
```

☰ Per ricapitolare >

I motivi principali per cui vengono usati le annotazioni di tipo sono:

- **Migliorare la leggibilità del codice:**
Le annotazioni di tipo aiutano a capire subito quale tipo di dato ci si aspetta per una variabile o una funzione, senza dover indovinare o eseguire il codice per verificarlo.
- **Supporto agli strumenti di analisi statica:**
Strumenti come mypy, PyCharm, o VS Code con Python linting attivato possono analizzare il codice e segnalare potenziali errori di tipo prima che il codice venga eseguito, migliorando la qualità del codice e riducendo gli errori.
- **Documentazione automatica:**
Le annotazioni di tipo servono anche come una forma di documentazione auto-descrittiva. Un altro sviluppatore che legge il codice può capire rapidamente che tipo di dati sono previsti senza dover esaminare tutto il contesto.
- **Migliorare l'auto completamento:**
Gli editor di codice e gli IDE moderni (come PyCharm o VS Code) possono usare le annotazioni di tipo per migliorare le funzionalità di auto completamento e suggerire i metodi disponibili per un determinato tipo.

!!!N.B: In Python, quando una variabile viene dichiarata, il suo tipo viene determinato automaticamente in base al valore che le viene assegnato. !!!

Una variabile può avere una sequenza di caratteri come valore:



Python

```
1  #stringa
2  stringa = "Questa è una stringa"
3  print(stringa)
4  print(type(stringa))
5
```

Una variabile può avere un valore booleano:



Python

```
1 #booleano
2 bln = True
3 print(bln)
4 print(type(bln))
```

In Python, quando una variabile viene dichiarata, il suo tipo viene determinato automaticamente in base al valore che le viene assegnato.



Python

```
1 a = 2.3
2 print(a)
3 print(type(a))
4
5 b = 10
6 print(b)
7 print(type(b))
8
9 c = 'io sono una stringa'
10 print(c)
11 print(type(c))
12
13
14 c1 = "io sono anche una stringa"
15 print(c1)
16 print(type(c1))
```

Quindi, in realtà, Python ha i suoi types, tuttavia sono collegati non ai nomi delle variabili ma agli oggetti stessi.

🔗 La funzione `type()` : ✓

La funzione `type` è una funzione integrata che serve per diversi scopi:

1. **Determinare il tipo di un oggetto:**

se passi un oggetto come argomento, `type()` restituisce la classe a cui l'oggetto appartiene.



Python

```
1 x = 5
2 print(type(x)) # output: <class 'int'>
```

1. Creare dinamicamente una nuova classe:

Se viene chiamata con tre argomenti, `type()` è utilizzato per creare una nuova classe.

La sintassi è:



Python

```
1 type(name, bases, dict)
```

- `name` : Nome della nuova classe (stringa).
- `bases` : Tupla delle classi base da cui ereditare.
- `dict` : Dizionario che contiene gli attributi e i metodi della classe.

Object-Oriented

Come accennato sopra, in python è tutto un oggetto.

Per capire meglio questo concetto riprendiamo dalla spiegazione che le variabili sono semplicemente dei puntatori e i nomi delle variabili stessi non hanno informazioni di tipo associate.

Ovviamente ciò porta l'utenza meno esperta a pensare che Python sia un linguaggio type-free, cosa che ovviamente non è e porta a numerosi errori. Prendendo ad esempio i seguenti casi:



Python

```
1 x = 4
2 print(type(x))
```

Output



Python

```
1 <class 'int'>
```



Python

```
1 x = 'hello'
```

```
2 print(type(x))
```

Output



Python

```
1 <class 'str'>
```



Python

```
1 x = 3.14
2 print(type(x))
```

Output



Python

```
1 <class 'float'>
```

Come si può notare, Python ha dei tipi; tuttavia, i types non sono collegati ai nomi delle variabili, ma agli oggetti stessi.

Nei linguaggi di programmazione orientati agli oggetti come Python, un oggetto è un'entità che contiene dati insieme a metadati e/o funzionalità associati.

In Python tutto è un oggetto, il che significa che ogni entità possiede alcuni metadati (chiamati attributi) e funzionalità associate (chiamate metodi).

Questi attributi e metodi sono accessibili tramite la sintassi del punto (.) ([Vedi le funzioni di python](#)).

Convenzioni di denominazione

Quando nomini le tue variabili, ci sono alcune cose che non puoi fare.

Ad esempio: gli spazi bianchi.

"alices age" non è una variabile valida! Invece, ad esempio: **alices_age** o **alicesAge**.

In generale:

- **Un nome di variabile deve iniziare con una lettera o il carattere di underscore (_).**
- **Un nome di variabile non può iniziare con un numero.**

- Un nome di variabile può contenere solo caratteri alfanumerici e underscore (A-z, 0-9, e _).
 - I nomi delle variabili sono sensibili al maiuscolo/minuscolo (ad esempio, age, Age e AGE sono tre variabili diverse).
 - Un nome di variabile non può essere una delle parole chiave di Python (come if, else, import, return, ecc.).
-

Strings

Una stringa è una sequenza di caratteri.

Quindi è possibile accedere a ogni carattere della stringa utilizzando un indice che indica la posizione di quel carattere all'interno della stringa data.



Python

```
1  Strings
2  #single line string
3  strA="Hello World«
4  #multiple line strings
5  straB="This is too \
6  bif to fit \
7  on a single line so \
8  you multi-lined it«
9  '''a string is a sequence of characters, so each character of the
   string can be accessed
10 using an index
11 that indicates the position of that character within the given
   string.'''
12 name="John Fitz«
13 print(name[0]) #display character at position 0 of string.
14 len(name) #return string lenght
15 print(len(name))
16 print(name.lower()) # Lowercase
17 print(name.upper()) # Uppercase
18 print(name.title()) # Title case
```
